

Lab08. Using AWS Secrets to pass secrets

Cloud Programming (W04IST–SI4527G)

Wojciech Thomas

Spring 2026

1 Learning goals

After completing this instruction you will be able to:

1. Configure secret in AWS.
2. Retrieve secret to AWS EC2 instance.
3. Use secret to access GitHub private repository.

2 Prerequisites

1. To complete this lab you have to create in advance Terraform module that creates EC2 instance. It will be referred as EC2 module.
2. Create a key pair (in Windows use Git Bash):

```
ssh-keygen -t ed25519 -C "mail@pwr.edu.pl" -f repo_key -N ""
```

Copy public and private keys (`repo_key` and `repo_key.pub`) into Terraform project folder.

3. Configure deploy key in your GitHub private repository [1].

3 Exercises

3.1 Create a new secret

1. In the `main.tf` file create a new resource to store secret value:

```
resource "aws_secretsmanager_secret" "github_key" {  
  name = "myproject/privkey"  
  recovery_window_in_days = 0  
}
```

It is crucial to set recovery window value to 0, as by default you can create a new secret with the same name after 30 days!

2. Add a new resource, to manipulate the content of the secret:

```
resource "aws_secretsmanager_secret_version" "github_key_value" {
  secret_id = aws_secretsmanager_secret.github_key.id
  secret_string = file("repo_key")
}
```

Command `file()` refers to the private key created as a prerequisite.

3. That is all. When you apply this configuration, Terraform will create a new secret, and store the private key inside.

3.2 Accessing secret from the EC2

1. In the `aws_instance` resource, locate argument `user_data`:

```
resource "aws_instance" "webserver" {
  ...
  # This resource contains arguments related to instruction only
  iam_instance_profile = "LabInstanceProfile"
  user_data = file("setup.sh")
}
```

The file `setup.sh` must be located in the same folder as Terraform file.

You have to configure `iam_instance_profile`, to give your EC2 instance permission to access Secrets Manager. In AWS Learners Lab you should always use `LabInstanceProfile`, as it gives you all required permissions.

Notice: In your own AWS account, you have to create your own profile that gives you access to the required resources only.

2. Add the following content to `setup.sh` script (this script is for Ubuntu/Debian Linux, in Amazon Linux replace user `ubuntu` with `ec2-user` and application `apt-get` with `dnf`):

```
#!/bin/bash
apt-get update
apt-get install -y awscli
HOME=/home/ubuntu
aws secretsmanager get-secret-value --region us-east-1 \
  --secret-id "myproject/privkey" \
  --query "SecretString" \
  --output text > $HOME/.ssh/repo_key.pem
chmod 600 $HOME/.ssh/repo_key.pem

cat > $HOME/.ssh/config <<- EOF
Host github.com
  Hostname github.com
  IdentityFile=~/.ssh/repo_key.pem
EOF

ssh-keyscan github.com >> $HOME/.ssh/known_hosts
chown ubuntu:ubuntu $HOME/.ssh/*
```

```
su - ubuntu -c "cd ; git clone git@github.com:user/privrepo.git"
```

In Ubuntu, you have to install `awscli` package. In Amazon Linux this package is installed by default.

Remember, that `user_data` is executed as `root`, so:

- you do not have to use `sudo`,
- when you create a private key file, you have to change the ownership from `root`, to your user,
- you should use absolute paths instead of `~` or `..`,
- the `git clone` command is run by user `ubuntu` (or `ec2-user` in Amazon Linux) using `su` command.

The command `ssh-keyscan` adds GitHub to the list of known hosts, so SSH can connect GitHub server without asking for permission.

3. When you create the EC2 instance, it will get the deployment key from Secrets Manager, and then clone the private repository from GitHub.

3.3 Challenge 1

1. In Terraform file, create a new AWS Secret containing GitHub Access Token.
2. At the end of the `script.sh`, add the code to get the Access Token from AWS Secret into your EC2 instance.
3. Add the code to clone private GitHub repo to EC2 instance, using Access Token for authorization.

You can clone remote repo to a new folder using the following syntax:

```
git clone https://github.com/user/privrepo.git newfolder/
```

3.4 Challenge 2

In this task exercise reuse the secret from the previous challenge.

1. Read AWS Secret into your Terraform script.
2. At the end of the `script.sh` file, add the placeholder for Access Token, e.g.:

```
GITHUB_TOKEN="%%PLACEHOLDER%%"
```

3. Use the Terraform `templatefile()` function, to replace a placeholder with the secret.
4. At the end of the `script.sh` file, add the code to clone your private repo into a new folder. Use the injected Access Token for authorization.

Bibliography

- [1] “Managing deploy keys.” [Online]. Available: <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/managing-deploy-keys/>.